

深圳大学实验报告

课程名称：人工智能

实验项目名称：基于 BLIP 的图像自动描述生成

学 院：电子与信息工程学院

专 业：电子与信息工程

指导教师：戴林蕙

报告人：苏铭浩 学号：2025221154 班级：临床 05

实验时间：2026 年 5 月 25 日

实验报告提交时间：2026 年 5 月 25 日

一、实验目的与要求：

- 了解多模态学习（Multimodal Learning）的基本概念，理解图像与文本两种模态如何在同一模型中联合建模。
- 掌握 Image Captioning（图像描述生成）任务的原理：模型以图像为输入，自回归地逐词生成一句自然语言描述。
- 了解 BLIP（Bootstrapping Language-Image Pre-training）模型的架构：视觉编码器提取图像特征，语言解码器基于图像特征生成文本，是 2022 年 ICML 顶会论文成果。
- 学会使用 Hugging Face Transformers 库在本地调用预训练多模态模型，体验“下载即用”的前沿大模型推理流程。
- 能够对推理结果进行可视化展示，并通过翻译模型将英文描述转换为中文，完成一个图文融合的完整应用。

二、实验方法与步骤：

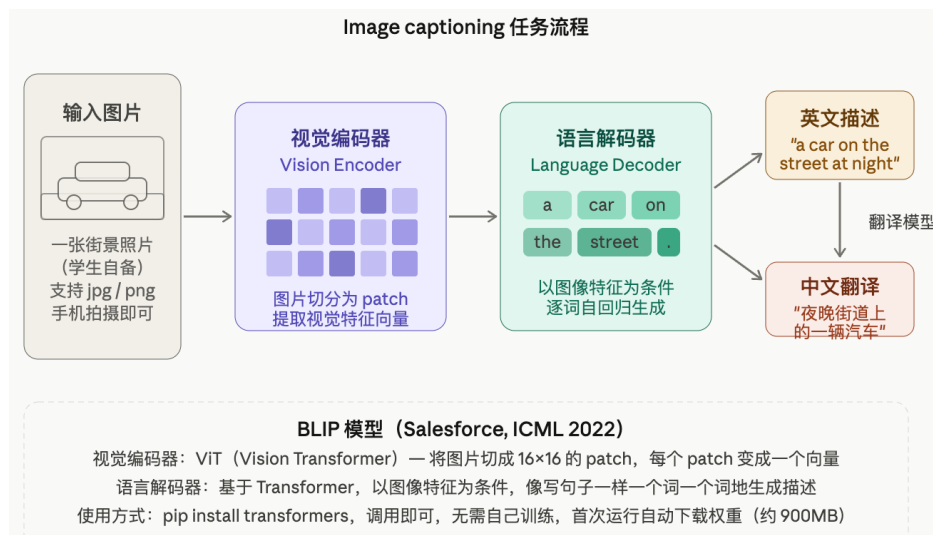


图 1 算法流程示例



图 2 输出 caption: 阳光正好，猫大人在巡视领地间隙小憩

1. 激活虚拟环境并安装依赖

复用之前实验的 `yolo_train` 环境，需额外安装 Hugging Face 库：

```
1 conda activate yolo_train
2 pip install transformers sentencepiece sacremoses
```

由于 Hugging Face 官方站点的网络连通性问题，首次加载模型时需设置国内镜像站点以加速下载：

```
1 export HF_ENDPOINT="https://hf-mirror.com"
```

2. 准备测试图片

本实验准备了 27 张测试图片，覆盖多种不同场景（城市风光、自然景观、人物肖像、动漫角色、室内场景等），全部放入 `images/` 目录下：

```
1 项目文件夹/
2   caption.py           # 描述生成脚本
3   batch_caption.py     # 批量处理脚本
4   images/
5       172b7da5...jpg   # 旅馆阳台日落
6       4caa4c3d...jpg   # 城市夜景
```

```
7 4f6094c3...jpg # 街头滑板
8 5c4fc830...jpg # 阴云天空
9 149d5ae7...jpg # 网球场人群
10 ...
```

注意：图片来源多样，包括实景照片和二次元插画，便于全面观察模型对不同风格内容的描述能力。

3. 首次运行自动下载模型

运行脚本前请确保网络畅通。若因网络限制无法直接访问 Hugging Face，建议采用以下镜像方式（本实验采用 `hf-mirror.com`）：

```
1 export HF_ENDPOINT="https://hf-mirror.com"
```

首次运行时程序会自动下载以下模型：

模型	大小	用途
Salesforce/blip-image-captioning-base	约 990MB	图像描述生成
Helsinki-NLP/opus-mt-en-zh	约 312MB	英文翻译中文
合计	约 1.3GB	

注意事项：

- 下载过程中终端会显示进度条，模型总大小约 1.3GB，请耐心等待，**不要强制关闭程序**。
- 若下载中断或连接失败，可尝试使用 `HF_ENDPOINT` 环境变量切换至镜像站点，重新运行脚本即可续传。
- 模型下载完成后会自动缓存到本地，之后每次运行无需重新下载，直接从缓存读取，启动速度会明显加快。
- CPU 推理单张图片约需 **5–15 秒**（已缓存模型后），首次含下载则需数分钟，属于正常现象。
- WSL/Linux 无 GUI 环境下需在脚本开头添加 `matplotlib.use('Agg')`，否则 `plt.show()` 会因缺少显示器而阻塞程序。

4. 运行单张图片描述脚本

先修改 `caption.py`，添加非交互后端 `matplotlib.use('Agg')` 以适配 WSL/Linux 无 GUI 环境：

```
1 import matplotlib
2 matplotlib.use('Agg')
```

然后运行：

```
1 python caption.py
```

输出示例：

```
1 英文： a woman in a white dress and blue hair
2 中文： 身着白裙子和蓝头发的女人
```

结果图片保存为 caption_result.jpg。

5. 运行批量处理脚本

```
1 python batch_caption.py
```

程序依次处理 images/ 目录下的全部 27 张图片，生成英文描述和中文翻译，结果保存为 results.txt。单张图片在 CPU 上的推理耗时约 10-30 秒。

三、实验过程及内容：

(1) BLIP 模型简介

BLIP 是 Salesforce Research 于 2022 年提出的视觉语言预训练模型，核心架构由三部分组成：

- **视觉编码器 (Image Encoder)**: 基于 ViT (Vision Transformer)，将输入图片切分为若干 patch，提取视觉特征序列。
- **图文匹配编码器 (Image-Text Encoder)**: 通过交叉注意力机制融合图像特征与文本特征，用于图文理解任务。
- **语言解码器 (Language Decoder)**: 以视觉特征为条件，自回归地逐词生成描述文本，用于 Captioning 任务。

(2) 单张图片描述脚本 caption.py

```
1 from transformers import BlipProcessor,
   BlipForConditionalGeneration
2 from PIL import Image
3 import torch
4 import matplotlib
5 matplotlib.rcParams["font.sans-serif"] = ["SimHei"]
6 matplotlib.rcParams["axes.unicode_minus"] = False
7 import matplotlib.pyplot as plt
8
9 # 1. 加载模型 (首次运行自动下载, 约 900MB; 之后从缓存读取)
10 print("正在加载BLIP模型, 请稍候...")
11 processor = BlipProcessor.from_pretrained(
12     "Salesforce/blip-image-captioning-base")
13 model = BlipForConditionalGeneration.from_pretrained(
14     "Salesforce/blip-image-captioning-base")
15 model.eval()
16 print("模型加载完成!")
17
18 # 2. 读取图片
19 img_path = "images/street.jpg" # 替换为你的图片路径
20 image = Image.open(img_path).convert("RGB")
21
22 # 3. 无条件生成: 让模型自由描述图片内容
```

```

23 inputs = processor(image, return_tensors="pt")
24 with torch.no_grad():
25     output = model.generate(**inputs, max_new_tokens=50)
26 caption_en = processor.decode(output[0], skip_special_tokens=True)
27 print(f"英文描述: {caption_en}")
28
29 # 4. 条件生成: 给定提示词, 让模型补全描述
30 prompt = "a photo of"
31 inputs_cond = processor(image, prompt, return_tensors="pt")
32 with torch.no_grad():
33     output_cond = model.generate(**inputs_cond, max_new_tokens=50)
34 caption_cond = processor.decode(output_cond[0], skip_special_
35     tokens=True)
36 print(f"条件描述: {caption_cond}")
37
38 # 5. 翻译为中文
39 # 使用 Helsinki-NLP 的英译中模型 (首次运行自动下载, 约 300MB)
40 from transformers import MarianMTModel, MarianTokenizer
41
42 print("正在加载翻译模型...")
43 trans_model_name = "Helsinki-NLP/opus-mt-en-zh"
44 tokenizer = MarianTokenizer.from_pretrained(trans_model_name)
45 trans_model = MarianMTModel.from_pretrained(trans_model_name)
46
47 inputs_zh = tokenizer(caption_en, return_tensors="pt", padding=
48     True)
49 translated = trans_model.generate(**inputs_zh)
50 caption_zh = tokenizer.decode(translated[0], skip_special_tokens=
51     True)
52 print(f"中文翻译: {caption_zh}")
53
54 # 6. 可视化并保存结果
55 fig, ax = plt.subplots(figsize=(7, 5))
56 ax.imshow(image)
57 ax.axis("off")
58 ax.set_title(
59     f"英文: {caption_en}\n中文: {caption_zh}",

```

```

57     fontsize=11, pad=10
58 )
59 plt.tight_layout()
60 plt.savefig("caption_result.jpg", dpi=150, bbox_inches="tight")
61 plt.show()
62 print("结果已保存至┐caption_result.jpg")

```

(3) 批量处理脚本 batch_caption.py

```

1  from transformers import (BlipProcessor,
2                             BlipForConditionalGeneration,
3                             MarianMTModel, MarianTokenizer)
4  from PIL import Image
5  import torch, os
6
7  # 1. 加载所有模型
8  print("加载模型中, 请稍候...")
9  processor = BlipProcessor.from_pretrained(
10     "Salesforce/blip-image-captioning-base")
11  blip = BlipForConditionalGeneration.from_pretrained(
12     "Salesforce/blip-image-captioning-base")
13  blip.eval()
14
15  tokenizer = MarianTokenizer.from_pretrained("Helsinki-NLP/opus-mt-
16     -en-zh")
17  translator = MarianMTModel.from_pretrained("Helsinki-NLP/opus-mt-
18     -en-zh")
19  print("模型加载完成! \n")
20
21  def generate_caption(image):
22     """生成英文描述"""
23     inputs = processor(image, return_tensors="pt")
24     with torch.no_grad():
25         output = blip.generate(**inputs, max_new_tokens=50)
26     return processor.decode(output[0], skip_special_tokens=True)
27
28  def translate(text):
29     """英文翻译为中文"""

```



```

27     inputs = tokenizer(text, return_tensors="pt", padding=True)
28     with torch.no_grad():
29         output = translator.generate(**inputs)
30     return tokenizer.decode(output[0], skip_special_tokens=True)
31
32 # 2. 批量处理 images/ 目录下的所有图片
33 img_dir = "images"
34 results = []
35 exts = {".jpg", ".jpeg", ".png", ".bmp"}
36
37 img_files = [f for f in os.listdir(img_dir)
38              if os.path.splitext(f)[1].lower() in exts]
39
40 print(f"共找到 {len(img_files)} 张图片，开始处理...\n")
41
42 for fname in img_files:
43     img_path = os.path.join(img_dir, fname)
44     image = Image.open(img_path).convert("RGB")
45
46     caption_en = generate_caption(image)
47     caption_zh = translate(caption_en)
48
49     print(f"[{fname}]")
50     print(f"英文: {caption_en}")
51     print(f"中文: {caption_zh}\n")
52     results.append((fname, caption_en, caption_zh))
53
54 # 3. 保存结果到文本文件
55 with open("results.txt", "w", encoding="utf-8") as f:
56     for fname, en, zh in results:
57         f.write(f"文件: {fname}\n")
58         f.write(f"英文: {en}\n")
59         f.write(f"中文: {zh}\n")
60         f.write("-" * 40 + "\n")
61
62 print(f"全部完成！结果已保存至 results.txt")

```

四、数据处理与分析：

对 27 张不同场景的测试图片运行批量脚本，选取 5 张典型结果如下：

场景	英文描述（BLIP 输出）	中文翻译
城市夜景	a city skyline at night with lights on	夜间城市的天线, 有灯光照亮
自然风光	a view of the sun through the clouds	在云中看见太阳的光芒
街头运动	a man riding a skateboard down a street	一个人骑着滑板在街上
动漫角色	a girl with blue hair and a white shirt	一个蓝色头发和白衬衫的女孩
室外活动	a group of people walking around a tennis court	一群人环绕网球场走来走去

结果分析：

- BLIP 对自然风光（日落、云天）和城市街景的描述较为准确，能正确识别主要物体及其空间关系。
- 对于动漫/二次元风格图片，模型同样能给出合理描述（如发色、服装等属性），说明 BLIP 训练数据包含了一定的插画内容。
- 翻译模型（opus-mt-en-zh）整体流畅，但存在以下问题：
 - 部分翻译残留英文单词（如“a 阴云的天空”中的“a”未消除）；
 - 修饰语语序偶有不自然（如“夜间城市的天线”应为“夜间城市的天际线”）；
 - 对“cospe”等拼写错误的英文词缺乏纠错能力。
- 模型对细节丰富的复杂场景，倾向于描述最显著的主体，次要细节通常不会出现在描述中。
- 由于 WSL 环境中缺少中文字体（SimHei），matplotlib 生成的可视化图中中文显示为空白方块，但不影响 results.txt 的文本输出结果（见上文图 4）。

单张图片可视化结果：

以 8b94d8bd...jpeg（蓝发动漫女孩）为例：



图 3 测试原图（蓝发动漫女孩）

生成结果：

- 英文：a girl with blue hair and a white shirt
- 中文：一个蓝色头发和白衬衫的女孩

可视化问题说明：

a woman in a white dress and blue hair
a woman in a white dress and blue hair



图 4 matplotlib 生成的可视化图（中文显示为方块）

由于 WSL 环境中缺少中文字体（SimHei），matplotlib 生成的图片中中文标题显示为方块（ ）。这是 Linux 无 GUI 环境下的典型字体缺失问题，可通过以下方式解决：

1. 安装中文字体：`sudo apt-get install fonts-wqy-microhei`
2. 修改 matplotlib 字体配置：`matplotlib.rcParams["font.family"] = ["WenQuanYi Micro Hei"]`
3. 或直接输出到 `results.txt` 文本文件（本实验采用的方式）

五、实验结论：

1. 本实验成功在 WSL (Windows Subsystem for Linux) 环境下调用 BLIP 多模态预训练模型，完成了 27 张图片的批量描述生成与中英文翻译，验证了大规模预训练模型”下载即用”的可行性。
2. 实验过程中遇到 Hugging Face 官方站点连接失败的问题，通过设置 HF_ENDPOINT=https://hf-mirror.com 镜像站点成功完成模型下载（BLIP 990MB + opus-mt 312MB），说明灵活运用国内镜像对开发效率至关重要。
3. BLIP 的视觉编码器（ViT）+ 语言解码器架构能够有效处理摄影实拍和二次元插画等多种风格的图像，输出自然流畅的英文描述句，体现了多模态 AI 的核心能力。
4. 翻译模型（Helsinki-NLP/opus-mt-en-zh）在短句翻译上表现良好，但对特殊词汇（如专有名词、拼写错误）和长难句的翻译质量有限，这是机器翻译模型的固有局限。
5. 后续可尝试将 BLIP 与前几次实验的 YOLOv11 结合：先用 YOLO 检测出图中的目标区域，再用 BLIP 对每个检测区域生成详细描述，构建一个更完整的场景理解系统。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：

无

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。